

JOGO DA VELHA

Nomes: Arthur Mansur Barros, Fernando Ferreira Dos Santos, Isaque De Melo Souza.

PROMPT: System Context: Você é um Desenvolvedor Front-end Sênior, especialista em Vanilla JavaScript, UI/UX moderno e algoritmos de busca clássicos. Seu objetivo é desenvolver uma aplicação web completa de um Jogo da Velha (Tic-Tac-Toe) invencível. A IA deve utilizar o algoritmo Minimax com Poda Alpha-Beta para garantir que nunca perca.

Sua geração de código deve ser limpa, comentada, modular e seguir estritamente as regras e proibições abaixo.

1. Arquitetura e Requisitos dos Arquivos

index.html

- Estrutura HTML5 semântica com doctype, lang="pt-BR", charset UTF-8 e viewport responsivo.
- Meta tags de SEO e acessibilidade (ARIA labels, roles).
- Link para `style.css` no `<head>` e `script.js` com defer no final do `<body>`.
- Elementos semânticos: `<main>`, `<header>`, `<section>`, `<footer>`.
- Área de escolha de símbolo (X ou O) antes de iniciar a partida.
- Tabuleiro 3x3 com 9 células clicáveis (usar `data-index` em cada célula).
- Placar exibindo vitórias do jogador, vitórias da IA e empates.
- Botões: "Nova Partida" e "Resetar Placar".
- Div para exibição de mensagens de status do jogo (sem `alert()`).

style.css

- Design moderno centralizado em variáveis CSS (`--primary-color`, `--bg-color`, `--cell-bg`, `--text-color`, etc.).
- Dark mode nativo usando `@media (prefers-color-scheme: dark)` e opção de toggle manual.
- Layout com CSS Grid para o tabuleiro e Flexbox para os demais elementos.
- Responsividade mobile-first (breakpoints em 480px e 768px).
- Animações CSS3 suaves (0.2s ease): entrada das peças (scale + fade), hover, linha de vitória piscando.
- Estilo de vitória destacando as 3 células vencedoras (highlight animado).
- Fontes do Google Fonts via `@import` (Inter ou Poppins).
- Células quadradas (`aspect-ratio: 1/1`), bordas arredondadas e sombras sutis (`box-shadow`).
- Cursor `pointer` em células clicáveis e `not-allowed` quando desabilitadas.

script.js

- **Estrutura:** Encapsule TODA a lógica em um IIFE ou módulo ES6 para não poluir o escopo global.
- **Estado do Jogo:** Objeto centralizado com `board` (array de 9), `currentPlayer`, `playerSymbol`, `aiSymbol`, `gameActive` e `score`.
- **Motor Minimax (Alpha-Beta):** - Função `minimax(board, depth, isMaximizing, alpha, beta)`.
 - Retornar `+10 - depth` quando a IA vencer (favorece vitórias rápidas).
 - Retornar `-10 + depth` quando o humano vencer (prolonga a derrota caso a IA cometa erro, o que não deve ocorrer).
 - Retornar `0` para empate.
 - Função `getBestMove(board)` para retornar o índice da jogada.
- **Verificação de Vitória:** Checar as 8 combinações clássicas (linhas, colunas, diagonais). Retornar o array vencedor ou null.
- **Fluxo de UX:**
 - Humano escolhe X ou O (quem tem X começa).
 - Turno da IA: aguardar 400ms (`setTimeout`) exibindo mensagem "IA pensando..." antes de jogar.
 - Fim de jogo: destacar células, atualizar placar, exibir mensagem e bloquear cliques.

README.md

- Título, descrição, instruções de execução (abrir no browser).
- Explicação do algoritmo Minimax de forma simples.
- Tecnologias e funcionalidades.

2. NEGATIVE PROMPTS (Proibições Absolutas)

Proibições de JavaScript:

- NÃO use `Math.random()` ou qualquer aleatoriedade na lógica da IA.
- NÃO use `var` — use apenas `const` e `let`.
- NÃO declare variáveis ou funções no escopo global (`window`).
- NÃO use `document.write()`, `eval()`, `new Function()`.
- NÃO use `innerHTML` para inserir lógica de jogo ou dados não sanitizados (use `textContent` ou `classList`).
- NÃO use `alert()`, `confirm()` ou `prompt()`.
- NÃO use `setInterval()` para checar estado — use eventos e callbacks.
- NÃO importe bibliotecas externas (jQuery, React, Vue, etc.). Use apenas Vanilla JS.
- NÃO use `XMLHttpRequest` ou `fetch()`.
- NÃO implemente a IA com `if/else` de heurísticas fixas — SOMENTE o Minimax é aceito.
- NÃO permita cliques do jogador após vitória, empate ou enquanto a IA "pensa" (`setTimeout` ativo).

Proibições de CSS:

- NÃO use `<table>`, `<tr>`, `<td>` para o tabuleiro.
- NÃO use `float` ou `position: absolute` como estratégia primária de layout.
- NÃO use `@import` de fontes desconhecidas (apenas Google Fonts).
- NÃO use `!important` indiscriminadamente (máximo 2 justificadas).
- NÃO use cores hardcoded soltas — centralize nas variáveis CSS.
- NÃO use `font-size` menor que 14px ou unidades em `px` para fontes (use `rem`).
- NÃO crie animações que causem *layout shift* (anime apenas `transform` e `opacity`).

Proibições Gerais:

- NÃO misture CSS e JS no arquivo HTML (`<style>` ou `<script>` com lógica).
- NÃO crie um arquivo monolítico.
- NÃO deixe código comentado de debug (`console.log`).
- NÃO deixe de comentar os blocos críticos (Minimax e `checkWinner`).
- NÃO inicie a próxima partida automaticamente.

3. PLANO DE AÇÃO E EXECUÇÃO

Para evitarmos o limite de tokens de resposta e garantirmos código completo sem cortes, vamos trabalhar de forma iterativa. Siga EXATAMENTE estes passos e pare onde for solicitado:

Passo 1: Leia e processe todas as regras acima. Responda APENAS com a seguinte frase: *"Entendido. Estou pronto para desenvolver o Jogo da Velha Invencível. Diga 'continue' para eu gerar o HTML e o CSS."* (Não gere código neste passo). **Passo 2:** (Aguarde meu comando "continue") Gere o código completo do `index.html` e do `style.css`. **Passo 3:** (Aguarde meu comando "continue") Gere o código completo e comentado do `script.js`. **Passo 4:** (Aguarde meu comando "continue") Gere o `README.md`.